EdgeVision project guide

Simple explanations and technical reference | Updated 29 October 2026

EdgeVision is a working smart camera prototype. It decides locally which frames deserve AI processing, confirms useful activity, and sends small event messages to AWS. This guide explains the experience in plain language, then the firmware, networking, integration, and testing behind it.

The current camera hardware is simulated on a Windows PC. The AWS backend is deployed. Ring integration has been tested against the official Developer Playground. Those are three separate parts of the system; a working cloud connection does not mean the code runs inside a physical Ring camera.

The idea in plain language

Think of EdgeVision as a careful receptionist for a camera. Movement gets its attention. A quick check discards frames that do not need further work. An object detector examines selected frames. Several positive checks turn a suspicion into a confirmed event. The cloud receives the event, rather than the video, through EdgeVision's event pipeline.

A typical result says that a person was detected at the front door, with a confidence score, a frame number, and timing information. A dashboard can display this without retrieving a recording. Events could support notifications or automations, although those downstream actions are not implemented here.

The practical goal is predictable resource use. A small device has limited memory and cannot run unlimited AI work. The prototype makes those limits visible, along with the behavior when a camera, network connection, or inference task fails. Battery benefits are plausible but have not been measured.

Use the redesigned application

The landing page introduces the system and shows footage from the existing evaluation clip. Follow one frame to explore the processing stages. Open demo leads to the workspace.

Recorded visit works without a running simulator. Play visit replays a captured firmware run with its actual telemetry and matching AWS event responses. Pause, resume, or start over. This mode does not send commands to your machine or AWS. The video is recorded evaluation footage, not a live Ring feed.

Live simulator controls the local running firmware. Connection settings provide the launch command and let you check the Wi-Fi module URL and events API separately. The browser cannot launch a native program, so you run the command in a repository terminal. The interface reports missing connections instead of inventing data.

The Demo view answers whether a trigger became a stored event. Its journey shows simulator readiness, camera wake, detection, and cloud storage. A previously stored event must not complete a new trigger. What changed shows captured frames, uploaded payload bytes, and the assumed video baseline. Latest result shows the backend result.

Events lists the selected source's event history. Filter detections or test events, refresh the live backend, and inspect the actual JSON payload. Engineering holds the frame chart, a table alternative, compute

budget, detection scores, tracked memory, buffer ownership, and recovery counters. Recovery lab exposes temporary faults only in live mode.

Start and test a live demo

From the repository root in PowerShell, build the simulator once:

```
.\firmware\build.ps1
```

Start the full demo using the deployed AWS backend and the local evaluation clip:

```
.\scripts\run_demo.ps1 -Aws -Clip eval\clips\testclip.mp4
```

The launch script starts the local page server, Wi-Fi module, and firmware. Open the local page at http://127.0.0.1:8000/. The script supplies the AWS events URL as a query parameter. Navigation preserves that API configuration.

Open demo, choose Live simulator, and check the connection. Send a test event in Engineering to verify storage without depending on object recognition. A selftest result should appear in Events. Trigger motion in Demo to wake the camera and process the local clip. A qualifying person, vehicle, or animal detection can produce a new stored event.

For a fully local backend, omit -Aws. The script then starts Mosquitto and the SQLite cloud stand-in. In connection settings, use http://127.0.0.1:8000/api/events as the events endpoint. Stop local demo processes with .\scripts\stop_demo.ps1. Stopping the local demo does not destroy the deployed AWS resources.

From motion to a stored event

A trigger arrives through the dashboard, the signed Ring webhook receiver, or the Ring API watcher. The gateway converts it into a localhost UDP MOTION command. The simulated hardware raises the motion interrupt. CameraTask wakes the camera and begins frame capture.

The virtual sensor reads the clip, models the sensor and image processing path, and provides grayscale frames to the firmware. DMA-style transfers include CRC integrity checks. A three-slot frame pool tracks ownership so capture and processing do not overwrite each other's buffers.

The governor admits work according to a token-bucket compute budget. Its role is to keep a backlog from growing when the device is busy. Admitted frames reach the motion gate. A cheap image comparison stops frames that do not justify inference, with periodic rechecks to avoid relying only on continuous movement.

InferenceTask submits selected work to the simulated neural accelerator. The host runs the NanoDet model through OpenCV. The accelerator delay is modeled at 75 ms for the FP32 configuration; it is not a physical chip measurement. Completion is signaled back to the firmware using a simulated interrupt.

DecisionTask checks recent results. A confidence score of at least 0.50 in three of five recent checks confirms a category. Freshness limits stop very old results from confirming new activity. A cooldown limits repeated notifications; a long visit can still generate several events. Person, vehicle, and animal are supported categories, but the recorded evaluation establishes only a limited person-visit scenario.

NetworkTask serializes the event into a compact message. A Python Wi-Fi module emulator handles MQTT and TLS outside the firmware. AWS IoT Core receives the event, an IoT rule writes it to

DynamoDB, and Lambda reads recent events for the HTTP API. The dashboard polls that API. Raw video is not sent through this event path.

The technical building blocks

FreeRTOS is the embedded operating system. Tasks divide responsibilities: capture, motion checking, inference, decision making, networking, telemetry, watchdog supervision, and fault handling. Queues pass work between tasks; notifications and simulated interrupts signal completion. Priorities and bounded queues make scheduling behavior explicit.

The firmware has a 512 KB heap budget. Its allocation tracking locks after startup. The evaluated runs used about 280 KB of tracked heap, with no later allocations. This figure does not include all PC memory used by OpenCV, the model, operating-system threads, or color-frame storage on the host. It is not a claim that the complete AI system runs in 280 KB.

The hardware abstraction layer provides the virtual camera, accelerator, time, input, and Wi-Fi link. This separates the firmware's behavior from its Windows simulator implementation. Porting to a real camera would require real sensor, accelerator, networking, and platform drivers plus hardware validation.

The network boundary is an AT-command-style link over localhost TCP. Firmware publishes with AT+PUB and receives publish acknowledgments. AT+TEL carries telemetry to the emulator. The local telemetry HTTP service sends updates to the dashboard with Server-Sent Events. Live controls send HTTP POST requests to the emulator, which forwards commands to the firmware.

The cloud stack is defined in TypeScript with AWS CDK. Python Lambda exposes GET /events with device and limit parameters. DynamoDB keys events by device and received_at. The IoT rule adds arrival time and an expires_at value for roughly 30-day retention; DynamoDB TTL deletion is asynchronous. The table uses on-demand capacity. Destroying the demo stack is configured to delete its event table and data.

The device certificate and private key are created outside CloudFormation and saved in the Git-ignored certs directory. The IoT policy restricts the device's client identity and publication topic. TLS and certificate handling live in the emulator. The current read API has no authentication and permits cross-origin GET requests, so stored event metadata is publicly readable.

Ring integration and its boundaries

The direct client in gateway/ring_api.py calls https://api.amazonvision.com using a Bearer token and the required User-Agent and Hardware_Id headers. It discovers accessible devices and reads event history. Tokens are entered at a hidden prompt or provided through RING_ACCESS_TOKEN. The client does not save tokens or place them in command-line arguments.

Run this alongside the demo to use the official Playground:

```
.\.venv\Scripts\python.exe gateway\ring_api.py --watch --playground
```

Generate a temporary token in the Ring Developer Playground and paste it at the hidden prompt. The Playground token expires after about 30 minutes. The watcher seeds existing history, then polls for new events every five seconds and suppresses duplicates. Restart it with a fresh token after expiry. With multiple accessible devices, select a Ring device ID using --device.

The Playground's Motion, Package, and Vehicle buttons create simulated live-view sessions. Their history uses event_type on_demand. Explicit --playground mode maps those new sessions to a clearly labeled local wake trigger. Normal --watch mode accepts motion events and does not treat an ordinary live-view session as a real motion alert. The current polling client examines the latest history page, so it is not a durable high-volume ingestion system.

The separate webhook gateway accepts Ring's webhook v1.1 structure. It verifies HMAC-SHA256 against the original request bytes before parsing, rejects invalid signatures, and deduplicates deliveries. It needs RING_HMAC_KEY and a public HTTPS endpoint registered with Ring. Actual Ring-to-public-webhook delivery has not been verified in this project.

The verified path is official Ring Playground event to API watcher to firmware wake to local-clip inference to AWS storage and retrieval. No physical Ring camera has been verified. Ring WHEP or RTSPS live video has not been connected to the virtual sensor. This prototype does not replace or install firmware on a Ring device.

Failures and recovery

Network outage: a bounded store-and-forward queue can hold up to 16 events. When connectivity returns, the firmware retries publication. This is an in-memory buffer, so it is not durable across process termination and cannot absorb unlimited outages.

Camera disconnect: the driver detects sensor failure and attempts recovery with increasing retry delays. Corrupt frame: a CRC mismatch causes the frame to be rejected rather than processed. Inference hang: the watchdog detects the unresponsive inference task and resets its simulated accelerator path. CPU overload: the governor drops more work to keep processing bounded. Memory pressure: the fault mechanism exercises behavior under constrained resources.

The Recovery lab exposes these scenarios, and Engineering displays the corresponding counters. Some faults are designed to make detection temporarily worse. A sent command only confirms that the simulator accepted the request; telemetry and later events establish the outcome. A network outage may need a new trigger or a test event to demonstrate queueing.

Evidence and known limits

The evaluation repeated one labeled person-visit clip six times and ran three empty-scene checks. All six clip runs detected the visit, with a median first event 4.5 seconds after the person appeared. There were no false events in those evaluated scenes. These results do not establish general recall or precision across new scenes, weather, lighting, people, or camera positions.

Median payload volume was 433 bytes per visit against an assumed 8.5 MB video baseline at 2 Mbit/s while awake. The reported saving concerns application payload, not total network traffic including MQTT, TCP, and TLS. Median AI workload was about 12 percent of captured frames. Roughly 77 percent stopped at the motion gate. The camera stayed awake for the whole visit in four of six runs; in other runs it could lose the visitor when they turned away.

The FreeRTOS Windows port is not a real-time hardware platform. Hardware delays are modeled. There are no measured battery, thermal, or physical accelerator results. Repeated alerts during a long visit, a finite outage queue, public event reads, and unverified physical Ring integration are current product limits.

The redesigned recorded demo uses a fresh actual simulator capture with matching AWS results. It is separate from the six-run benchmark. Its controls replay recorded observations and do not simulate successful commands to a live device.

Repository map and verification

firmware contains the C and C++ FreeRTOS application, virtual hardware, and firmware tests. netmodule contains the Python MQTT and telemetry emulator. gateway contains the signed webhook receiver and direct Ring API client. cloud/aws contains CDK and Lambda. cloud/local contains the Mosquitto subscriber, SQLite store, and static page server. dashboard contains the landing page, recorded and live workspace, and this web guide. eval contains evaluation scripts and results. scripts contains build and demo lifecycle commands.

Run .\firmware\test.ps1 for the firmware and gateway suite, or add -Quick for the fast subset. From gateway, run ..\.venv\Scripts\python.exe -m unittest -v test_ring_api test_ring_gateway for the focused Ring tests. Run node --test tests/flow.test.cjs for frontend state and navigation rules. Browser verification should cover replay and reset, source switching, stale connections, event filters and payload inspection, command failures, preserved API URLs, phone layouts, keyboard use, and reduced motion.

The main evidence sources are README.md, eval/results.md, docs/ring-notes.md, the firmware configuration and task implementations, the CDK stack, and the verified live API responses. Ring API behavior is documented at https://developer.amazon.com/docs/ring/api-documentation.html. The deployed backend uses ap-southeast-2 and AWS CLI profile edgevision.